

RIPPLEWEALTH - AN AI-DRIVEN INVESTMENT RISK INTELLIGENCE PLATFORM

API DOCUMENTATION

Version: 1.0.0

Base URL: <http://127.0.0.1:8000>

Architecture: FastAPI + Multi-Agent AI System

Authentication: Bearer Token (JWT)

1. INTRODUCTION

RippleWealth is an AI-driven financial analytics platform designed to analyze user investment portfolios and provide intelligent risk insights. The system leverages a multi-agent architecture to perform risk analysis, Monte Carlo simulations, causal inference, and real-time market event detection.

This API enables developers to:

- Upload and manage portfolios
- Perform advanced risk analysis
- Retrieve real-time market insights
- Interact with an AI-powered financial advisor

2. AUTHENTICATION & SESSION MANAGEMENT

RippleWealth uses JWT (JSON Web Token) for secure authentication.

◆ POST /auth/login

Description: Authenticates a user and returns an access token.

Request Body: `</>` JSON

```
{
  "username": "victoria_franklin",
  "password": "your_secure_password"
}
```

Response (200 OK): `</>` JSON

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

3. PORTFOLIO MANAGEMENT

Handles structured portfolio data stored in PostgreSQL.

◆ POST /portfolio/upload

Description: Uploads a user's investment portfolio for analysis.

Headers:

Authorization: Bearer <access_token>

Request Body: </> JSON

```
{
  "portfolio_name": "Tech_Growth_2026",
  "assets": [
    {"ticker": "AAPL", "quantity": 10, "avg_price": 185.50},
    {"ticker": "TSLA", "quantity": 5, "avg_price": 170.20}
  ]
}
```

Response (200 OK): </> JSON

```
{
  "message": "Portfolio uploaded successfully"
}
```

◆ GET /portfolio/list

Description: Retrieves all portfolios for the authenticated user.

Response: </> JSON

```
[
  {
    "portfolio_id": "uuid",
    "portfolio_name": "Tech_Growth_2026"
  }
]
```

◆ DELETE /portfolio/{id}

Description: Deletes a specific portfolio.

Path Parameter:

- `id` – Portfolio ID

Response: `</>` JSON

```
{
  "message": "Portfolio deleted successfully"
}
```

4. RISK ENGINE & INTELLIGENCE LAYER

Triggers the multi-agent orchestration system.

◆ **GET /analyze/{user_id}**

Description: Executes full AI pipeline including risk analysis, simulations, and causal mapping.

Path Parameter:

- `user_id` – Unique user identifier

Response (200 OK): `</>` JSON

```
{
  "risk_metrics": {
    "VaR": -0.042,
    "Sharpe_Ratio": 1.85,
    "volatility": "Moderate"
  },
  "simulations": {
    "paths": 1000,
    "expected_return": "12.4%"
  },
  "causal_graph": "json_graph_data_string"
}
```

◆ **POST /simulation/run**

Description: Runs a customized Monte Carlo simulation using user-defined stress parameters.

Headers:

Authorization: Bearer <access_token>

Request Body: </JSON>

```
{
  "portfolio_id": "uuid",
  "stress_factor": "inflation",
  "value": 0.05,
  "iterations": 5000
}
```

Response (200 OK): </JSON>

```
{
  "status": "Simulation completed",
  "expected_return": "10.2%",
  "risk_level": "High"
}
```

5. BUTTERFLY EVENT DETECTION (REAL-TIME)

Detects global events and their financial impact.

◆ GET /market/headlines

Description: Fetches real-time global events and classifies their financial impact.

Response (200 OK): </> JSON

```
{
  "event_id": "EVT_9921",
  "headline": "Regional Power Grid Disruption in SE Asia",
  "impact_score": "High",
  "affected_sectors": ["Semiconductors", "Supply Chain"]
}
```

6. REAL-TIME UPDATES (WEB-SOCKET)

◆ WS /ws/updates

Description: Establishes a WebSocket connection to stream real-time market events and risk alerts from the AI system to the frontend dashboard.

Connection URL:

ws://127.0.0.1:8000/ws/updates

Data Payload (Example): </JSON>

```
{
  "event_id": "EVT_1023",
  "headline": "Global Oil Supply Shock",
  "impact_score": "High"
}
```

Notes:

- Used for live updates (no manual refresh required)
- Powered by [agent_orchestrator.py](#)
- Consumed by frontend dashboard (React)

7. NATURAL LANGUAGE ADVISOR

AI-powered financial explanation and recommendation system.

◆ **POST /advisor/query**

Description: Allows users to query portfolio risk using natural language.

Request Body: </> JSON

```
{
  "query": "How does the power grid failure affect my Apple holdings?"
}
```

Response (200 OK): </> JSON

```
{
  "advice": "The power grid failure increases supply chain risk for AAPL's regional suppliers by 15%. Recommend hedging with defensive sectors."
}
```

8. SYSTEM ARCHITECTURE OVERVIEW

RippleWealth uses a multi-agent AI system, including:

- **Risk Analysis Agent** – Calculates financial metrics
- **Simulation Agent** – Runs Monte Carlo simulations
- **Causal Agent** – Builds event-impact relationships
- **Explanation Agent** – Generates human-readable insights

9. HOW TO TEST USING POSTMAN

Step 1: Setup

- Create a new collection: **RippleWealth API**
- Set base URL:
http://127.0.0.1:8000

Step 2: Authentication

1. Run `/auth/login`
2. Copy `access_token`
3. Go to Authorization tab → Select **Bearer Token**
4. Paste token

Step 3: API Testing Flow

Run APIs in this sequence:

1. Login
2. Upload Portfolio
3. Analyze Portfolio
4. Fetch Market Headlines
5. Advisor Query

10. ERROR HANDLING

</JSON>

{

"error": "Invalid token",

```
"status": 401
```

```
}
```

11. SYSTEM HEALTH MONITORING

◆ GET /health

Description: Checks API status and server health.

Response: </JSON>

```
{
```

```
"status": "OK",
```

```
"service": "RippleWealth API"
```

```
}
```

12. CONCLUSION

This API enables seamless integration of AI-driven financial analytics, allowing users to gain deep insights into portfolio risks, market events, and investment strategies through an intelligent, scalable system.